Algorithm2: *Problem Solving* with *Algorithms* and *Data Structures*

Lecture 04.

동적 계획법 (1)

경북대학교 배준현 교수
(joonion@knu.ac.kr)

Lec. 04. 동적 계획법 (1)

동적 계획법: *Dynamic Programming*

분할정복처럼 주어진 문제를 더 작은 부분 문제로 쪼개어 해결하는 알고리즘 설계 기법이지만,

중복되는 부분 문제를 두 번 이상 계산하는 비효율적인 상황이 발생하지 않도록,

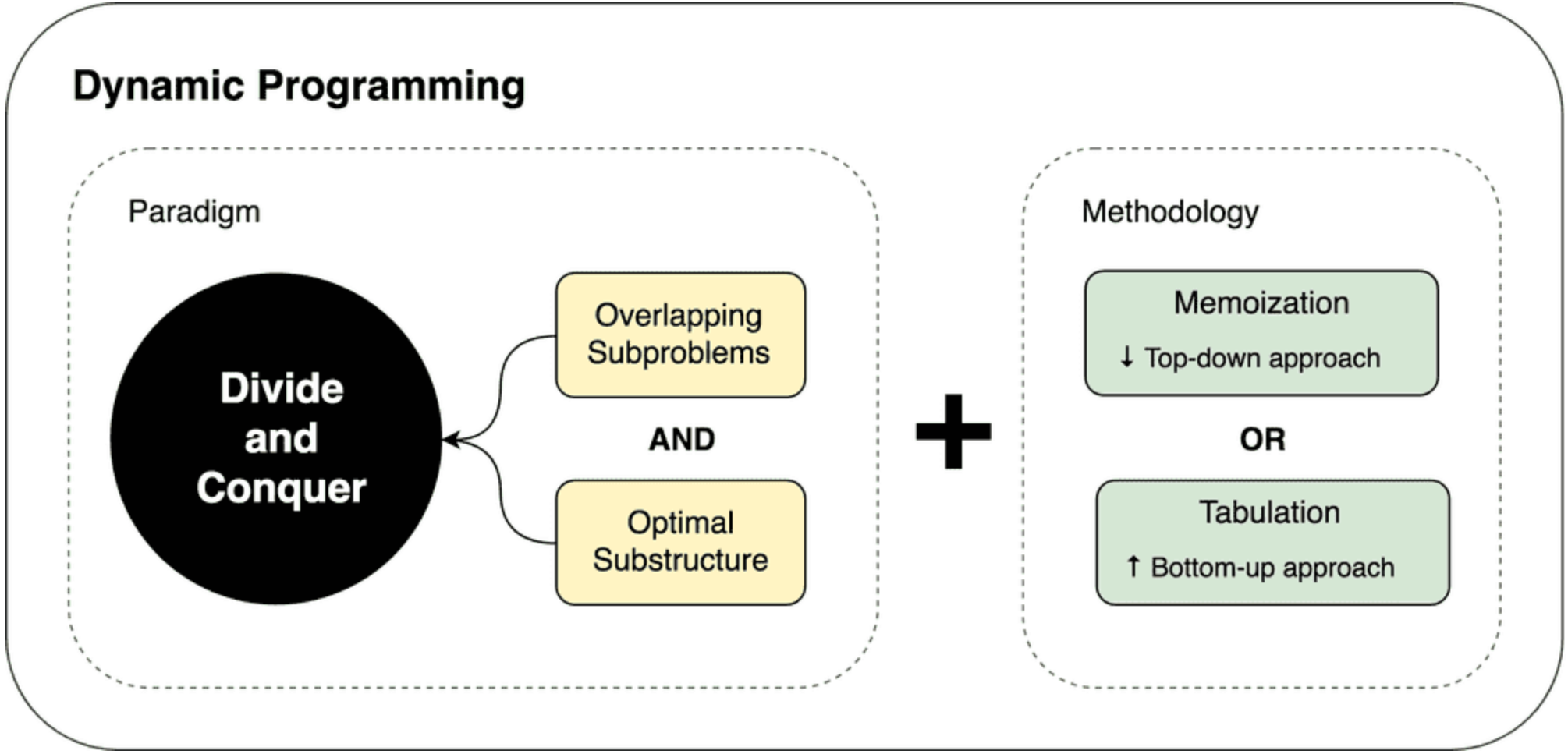
부분 문제를 해결한 결과를 별도의 메모리(*cache*, *table*)에 저장해 두고 재활용함

분할정복처럼 재귀적 관계에 기반하여 문제를 해결하지만,

메모이제이션 기법을 이용하여 중복되는 부분 문제를 해결하거나 (재귀 구조, 하향식 접근),

태뷸레이션 기법을 이용하여 더 작은 부분 문제를 먼저 해결함 (반복 구조, 상향식 접근)

Lec. 04. 동적 계획법 (1)



https://trekhleb.dev/blog/2018/dynamic-programming-vs-divide-and-conquer/

동적 계획법 레시피

1. 주어진 문제를 완전 탐색(주로 분할 정복)을 이용해 해결한다.
    1) 중복 부분 문제가 발생하는가?
    2) 최적 부분 구조가 보장되는가?

2. 중복되는 부분 문제를 한 번만 계산하도록 동적계획법을 적용한다.
    1) 주어진 문제와 부분 문제와의 재귀적 관계를 파악한다.
    2) 부분 문제를 저장할 저장 공간(테이블)의 형태를 결정한다.
    3) 메모이제이션 또는 태뷸레이션으로 동적계획법 알고리즘을 구현한다.

중복 부분 문제: *overlapping subproblem*

어떤 부분 문제가 두 개 이상의 다른 부분 문제를 푸는데 사용되는 경우

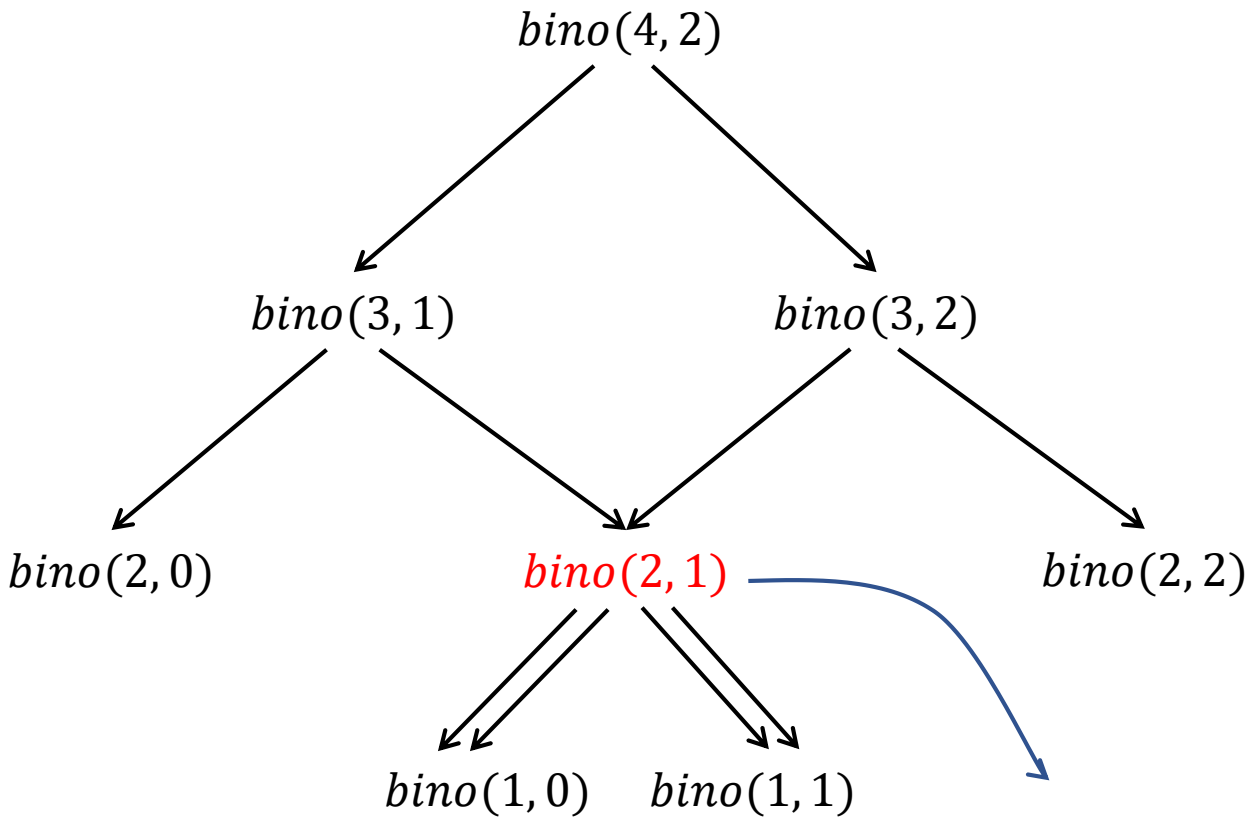
예) 이항 계수: *binomial coefficient*

$$\binom{n}{k} = 1, k = 0 \; or \; n = k$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

```
# 이항 계수의 재귀적 구현: 중복 부분 문제 발생
def bino(n, k):
    if k == 0 or n == k: # 종료 조건
        return 1
    else: # 재귀 조건
        return bino(n - 1, k - 1) + bino(n - 1, k)
```

Lec. 04. 동적 계획법 (1)



$bino(2,1)$은 $bino(3,1)$과 $bino(3,2)$가 중복해서 호출한다.

메모이제이션: *memoization*

함수의 결과를 저장하는 장소를 따로 마련해 두고,

한 번 계산한 값은 저장해 두었다가 이를 재활용하는 최적화 기법

```
# 메모이제이션을 위한 캐시 테이블
cache = [[-1] * (i + 1) for i in range(n + 1)]
```

```
# 메모이제이션: 별도의 캐시에 결과값을 저장
def bin2(n, k):
    global cache # 캐시는 주로 전역 변수로 선언
    if k == 0 or n == k:
        return 1
    elif cache[n][k] != -1: # 이미 계산된 결과가 있으면
        return cache[n][k] # 계산된 결과를 재활용
    else: # 아니면 재귀 호출로 구한 값을 캐시에 저장
        cache[n][k] = bin2(n - 1, k - 1) + bin2(n - 1, k)
        return cache[n][k]
```

Lec. 04. 동적 계획법 (1)

태뷸레이션: *tabulation* (*iterative dynamic programming*)

부분 문제 간의 재귀적 관계를 파악하기 쉬울 경우

더 작은 하위 부분 문제부터 시작하여 상위 부분 문제로 반복문을 사용하여 구현

```
# 태뷸레이션: DP 테이블을 구성하고 상향식으로 계산
def bin3(n, k):
    dp = [[-1] * (i + 1) for i in range(n + 1)] # 파스칼의 삼각형
    for i in range(n + 1):
        for j in range(min(i, k) + 1):
            if j == 0 or j == i:
                dp[i][j] = 1
            else: # 테이블의 하위에서 미리 계산된 값으로 상위에 있는 값을 계산
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
    return dp[n][k]
```

메모이제이션(재귀적 동적계획법)

장점:

1. 좀 더 직관적인 코드를 짤 수 있다.
2. 부분 문제 간의 의존 관계나 계산 순서에 대해 고민할 필요가 없다.
3. 전체 부분 문제 중 일부의 답만 필요할 경우 더 빠르게 동작한다. 예) 0 – 1 Knapsack Problem

단점:

1. 슬라이딩 윈도우 기법을 적용하기 어렵다.
2. 스택 오버플로우를 조심해야 한다.

태뷸레이션(반복적 동적 계획법)

장점:

1. 구현이 대개 더 짧다.
2. 재귀 호출에 필요한 부하가 없기 때문에 조금 더 빠르게 동작한다.
3. 슬라이딩 윈도우 기법으로 최적화하기에 좋다. (공간 복잡도를 개선할 수 있다.)

단점:

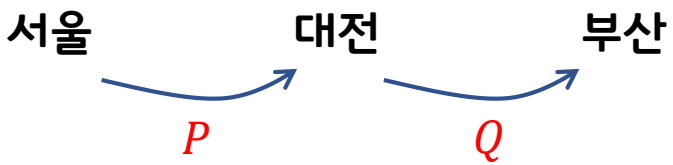
1. 구현이 좀 더 비직관적이다.
2. 부분 문제 간의 의존 관계를 고려해 계산되는 순서를 고려해야 한다.

Lec. 04. 동적 계획법 (1)

최적 부분 구조: *optimal substructure*

각 부분 문제의 최적해만 계산할 수 있으면 전체 문제의 최적해를 계산할 수 있는 조건

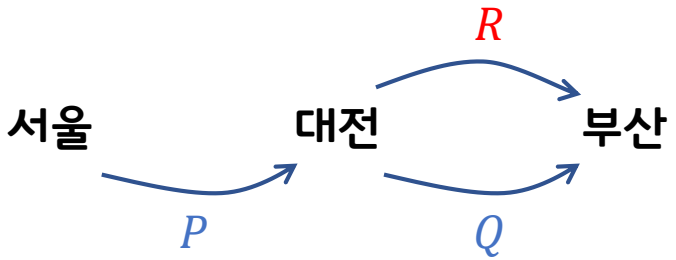
서울에서 부산까지 가는 최단 경로가
대전을 거쳐간다고 가정하면



경로 P가 (서울,대전)의 최단 경로이고,
경로 Q가 (대전, 부산)의 최단 경로이면,
두 경로를 합치면 항상 (서울,부산)의 최단 경로이다.

따라서, 최적 부분 구조가 성립한다.

서울에서 부산까지 가는 최단 경로를 찾고 싶은데
통행료의 합이 3만원을 초과하지 않아야 한다면



경로 Q: 통행시간 1시간, 통행료 2만원
경로 R: 통행시간 2시간, 통행료 1만원
경로 R과 Q의 선택은 경로 P의 통행료에 따라 달라진다.

따라서, 최적 부분 구조가 성립하지 않는다. (DP 적용 불가)

AL2.4.1: 삼각형 위의 최대 경로 (ID: TRIANGLEPATH)

문제 아래 형태와 같이 삼각형 모양으로 배치된 자연수들이 있습니다. 맨 위의 숫자에서 시작해, 한 번에 한 칸씩 아래로 내려가 맨 아래 줄로 내려가는 경로를 만들려고 합니다. 경로는 아래 줄로 내려갈 때마다 바로 아래 숫자, 혹은 오른쪽 아래 숫자로 내려갈 수 있습니다. 이 때 가능한 모든 경로 중 경로에 포함된 숫자의 최대 합을 찾는 프로그램을 작성하세요.

```
6
1  2
3  7  4
9  4  1  7
2  7  5  9  4
```

Lec. 04. 동적 계획법 (1)

입력 입력의 첫 줄에는 테스트 케이스의 수 C(C <= 50)가 주어집니다. 각 테스트 케이스의 첫 줄에는 삼각형의 크기 n(2 <= n <= 100)이 주어지고, 그 후 n줄에는 각 1개~n개의 숫자로 삼각형 각 가로줄에 있는 숫자가 왼쪽부터 주어집니다. 각 숫자는 1 이상 100000 이하의 자연수입니다.

출력 각 테스트 케이스마다 한 줄에 최대 경로의 숫자 합을 출력합니다.

예제 입력

```
5
6
1 2
3 7 4
9 4 1 7
2 7 5 9 4
```

예제 출력

```
28
```

완전 탐색(분할 정복)으로 시작하기

종료 조건: 맨 아래쪽까지 내려온 경우

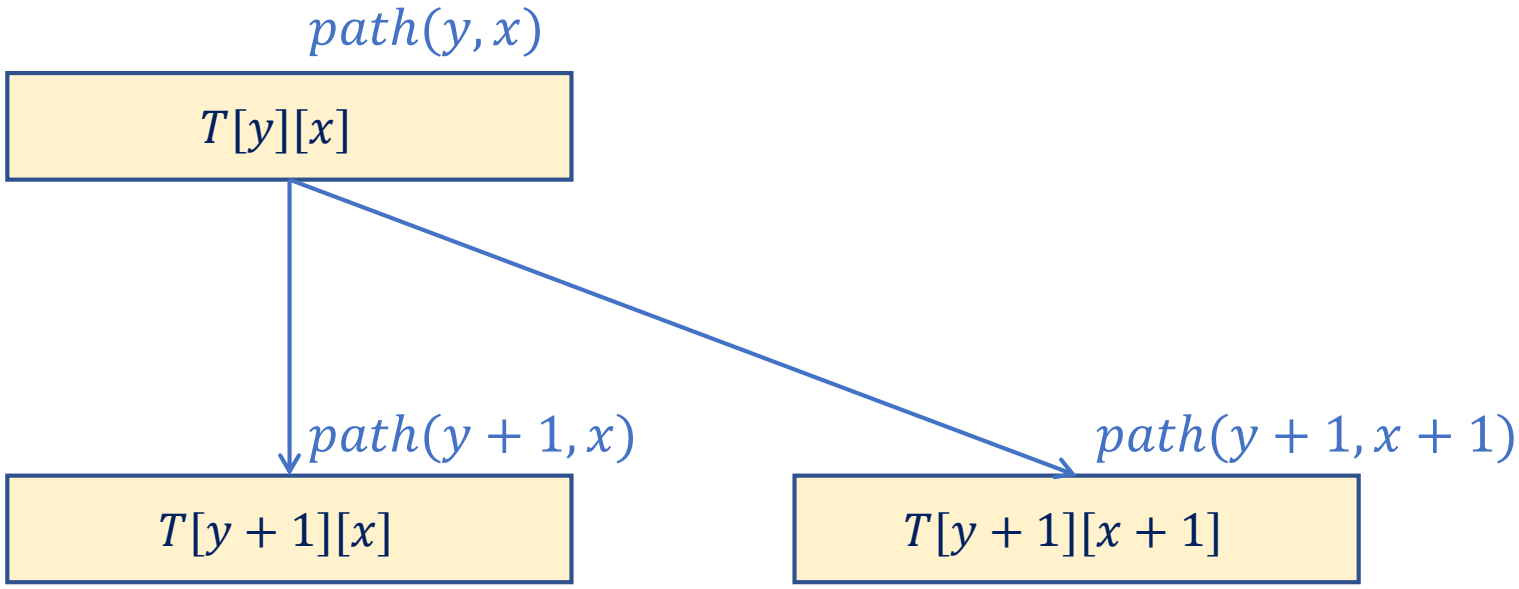
해당하는 칸(y, x)에 있는 값$(T[y][x])$를 반환하고 종료

재귀 조건: 바로 아래로 내려가거나, 오른쪽 아래로 내려가야 함

(바로 아래로 내려간 칸에서의 최적값, 오른쪽 아래로 내려간 칸에서의 최적값)

둘 중에서 더 큰 값을 현재 칸에 있는 값과 더하면 현재 칸에서 얻을 수 있는 최대 경로의 합

$path(y, x)$: (y, x)에서 시작해서 맨 아래줄까지 내려가는 부분 경로의 최대 합



$$path(y, x) = T[y][x] + \max\big(path(y + 1, x), path(y + 1, x + 1)\big)$$

$$path(n - 1, x) = T[y][x]$$

Lec. 04. 동적 계획법 (1)

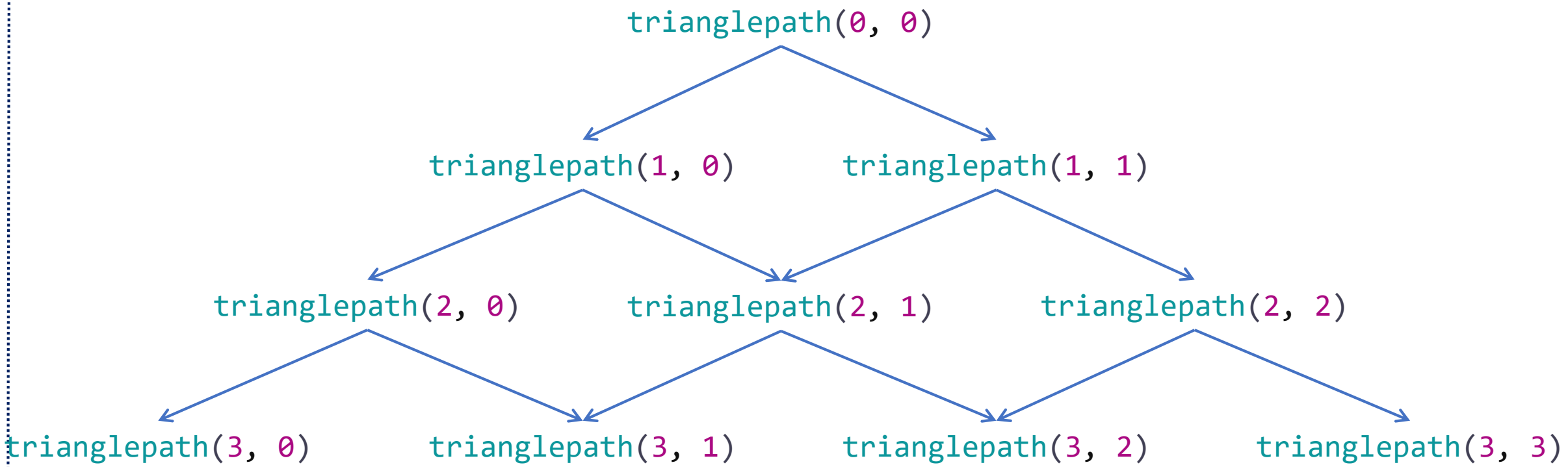
```
'''
P.4.1. 삼각형 위의 최대 경로 (ID: TRIANGLEPATH): 분할 정복
'''
for _ in range(int(input())):
    n = int(input()) # 삼각형의 크기
    T = [list(map(int, input().split())) for _ in range(n)] # 삼각형의 숫자
    print(trianglepath(0, 0))
```

$(y = 0, x = 0)$**에서 최대 경로의 합을 구한다.**

Lec. 04. 동적 계획법 (1)

```python
# 분할 정복으로 해결하기
def trianglepath(y, x):
    global n, T
    if y == n - 1: # 삼각형 맨 아래에 도착하면
        return T[y][x]
    else:
        down = trianglepath(y + 1, x) # 바로 아래로 갈 때의 최적값
        right = trianglepath(y + 1, x + 1) # 오른쪽 아래로 갈 때의 최적값
        return T[y][x] + max(down, right)
```

Lec. 04. 동적 계획법 (1)



중복 부분 문제가 발생함

메모이제이션으로 해결하기

$T =$

6				
1	2			
3	7	4		
9	4	1	7	
2	7	5	9	4

$cache =$

-1				
-1	-1			
-1	-1	-1		
-1	-1	-1	-1	
-1	-1	-1	-1	-1

메모이제이션으로 해결하기

$T =$

6				
1	2			
3	7	4		
9	4	1	7	
2	7	5	9	4

$cache =$

28				
20	22			
19	18	20		
16	11	10	16	
-1	-1	-1	-1	-1

Lec. 04. 동적 계획법 (1)

```
# 메모이제이션을 위한 캐시 테이블
cache = [[-1] * (i + 1) for i in range(n)]

# 메모이제이션으로 해결하기
def trianglepath(y, x):
    global n, T, cache
    if y == n - 1:
        return T[y][x]
    elif cache[y][x] != -1: # 이미 캐시에 저장된 값이 있으면
        return cache[y][x] # 재활용
    else:
        down = trianglepath(y + 1, x)
        right = trianglepath(y + 1, x + 1)
        cache[y][x] = T[y][x] + max(down, right) # 계산 결과를 캐시에 저장해 두고
        return cache[y][x] # 계산 결과 반환
```

태뷸레이션으로 해결하기

$T =$

6				
1	2			
3	7	4		
9	4	1	7	
2	7	5	9	4

$dp =$

-1				
-1	-1			
-1	-1	-1		
-1	-1	-1	-1	
2	7	5	9	4

맨 아래줄 최적값은 T에서 가져와서 초기화

태뷸레이션으로 해결하기

$dp[0][0]$: 맨 위에서 맨 아래쪽까지 내려갔을 때의 최적값

$T =$

6				
1	2			
3	7	4		
9	4	1	7	
2	7	5	9	4

$dp =$

28				
20	22			
19	18	20		
16	11	10	16	
2	7	5	9	4

bottom-up

```
print(trianglepath(n, T))


# 태뷸레이션으로 해결하기
def trianglepath(n, T):
    # 태뷸레이션을 위한 dp 테이블 생성
    dp = [[-1] * (i + 1) for i in range(n)]
    # 맨 아래줄 정보는 T로부터 바로 초기화
    dp[n - 1] = T[n - 1][:] # shallow copy
    # 맨 아래줄 바로 위부터 bottom-up
    for i in range(n - 2, -1, -1):
        for j in range(i + 1):
            dp[i][j] = T[i][j] + max(dp[i + 1][j], dp[i + 1][j + 1])
    return dp[0][0] # 최적값은 dp 테이블의 (0, 0) 위치에 있음
```

태뷸레이션 + 공간 복잡도 개선: 슬라이딩 윈도우 적용하기

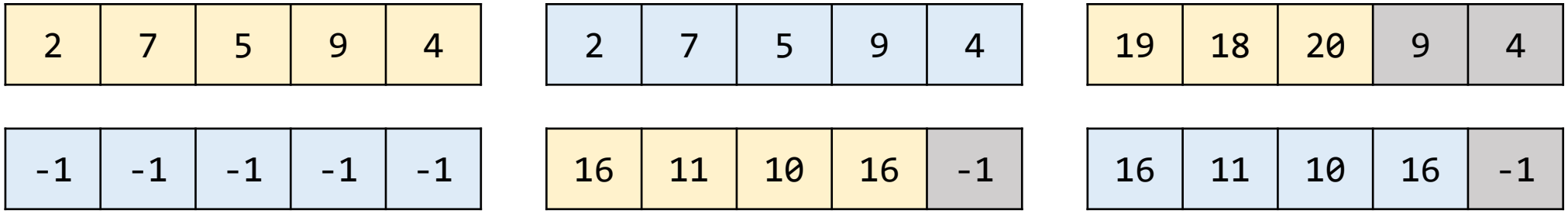
$n = 10{,}000$인 경우: 공간 복잡도가 $O(n^2)$이므로, 메모리 초과!

슬라이딩 윈도우를 적용하여 공간 복잡도를 한 차수 낮출 수 있다.

재귀식: $dp[i][j] = T[i][j] + \max(dp[i+1][j], dp[i+1][j+1])$

i번째 가로줄 $dp[i]$를 계산하려면, $i+1$번째 가로줄 $dp[i+1]$만 필요하다.

두 개의 윈도우를 생성하여, 번갈아 가며 사용한다.

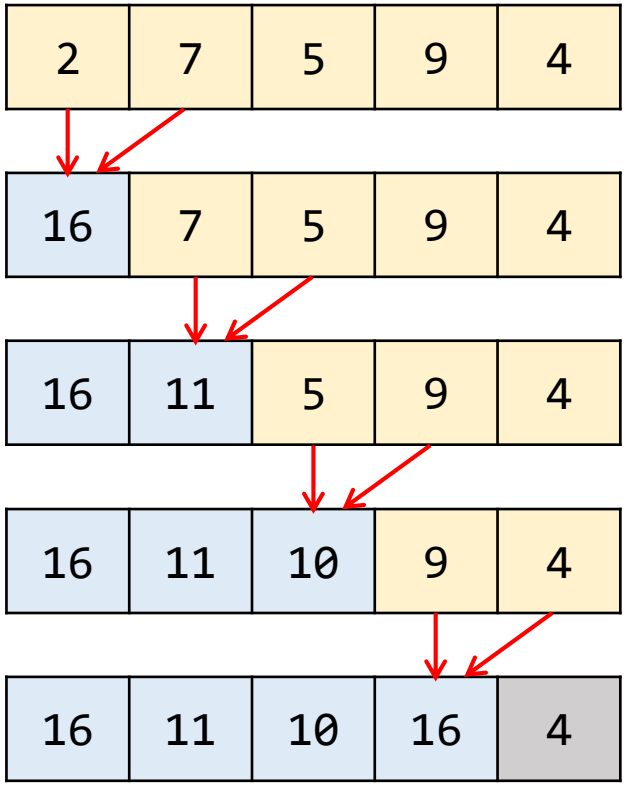


```python
# 태뷸레이션 + 공간 복잡도 개선 (슬라이딩 윈도우 적용)
def trianglepath(n, T):
    # 슬라이딩을 위한 윈도우 두 개 생성
    dp = [[-1] * n for _ in range(2)]
    # 맨 아래줄 정보는 T로부터 바로 초기화
    dp[(n-1)%2] = T[n - 1][:] # shallow copy
    # 맨 아래줄 바로 위부터 bottom-up으로 슬라이딩
    for i in range(n - 2, -1, -1):
        for j in range(i + 1):
            dp[i%2][j] = T[i][j] + max(dp[(i+1)%2][j], dp[(i+1)%2][j + 1])
    return dp[0][0] # 최적값은 슬라이딩 윈도우의 (0, 0) 위치에 있음
```

Lec. 04. 동적 계획법 (1)

그런데, 윈도우가 반드시 두 개가 필요한가?

현재 윈도우의 값을 계산할 때, 먼저 계산된 값이 다른 위치의 값에 영향을 주지 않으면?



j번째 원소의 값을 계산할 때

j번째 원소의 값과 $j+1$번째 원소의 값만 고려한다.

j번째 원소의 값이 변경된 이후의 계산에서

j번째 원소의 값은 다시 고려하지 않는다.

따라서, 한 개의 윈도우 만으로도 충분하다.

※ 이항 계수($bin3$)에도 이 방법을 적용해보고

한 개의 윈도우를 사용하는 트릭을 찾아보시오.

Lec. 04. 동적 계획법 (1)

```python
# 태뷸레이션 + 공간 복잡도 개선 (하나의 윈도우)
def trianglepath(n, T):
    # 슬라이딩을 위한 윈도우 한 개 생성
    # 맨 아래줄 정보는 T로부터 바로 초기화
    dp = T[n - 1][:] # shallow copy
    # 맨 아래줄 바로 위부터 bottom-up으로 슬라이딩
    for i in range(n - 2, -1, -1):
        for j in range(i + 1):
            dp[j] = T[i][j] + max(dp[j], dp[j + 1])
    return dp[0] # 최적값은 슬라이딩 윈도우의 0번째 위치에 있음
```

삼각형 위의 최대 경로 문제: 최적 부분 구조가 성립하는가?

삼각형 위의 최대 경로 문제는 최대 경로의 합을 찾는 최적화 문제이다.

이 문제의 재귀적 관계에서 현재 문제는 두 개의 부분 문제로 분할한다.

각 부분 문제의 최적값을 구하면, 둘 중에서 더 큰 값을 선택하여 현재 값과 더한다.

부분 문제의 최적해로 전체 문제의 최적해를 알 수 있으므로,

최적 부분 구조가 성립한다.

functools의 @lru_cache, @cache 데코레이터 활용

재귀 함수의 동작을 확인하고자 할 때, @cache 데코레이터를 활용할 수 있다.

@cache 데코레이터는 모든 함수의 결과를 자동으로 캐싱해주는 데코레이터이다. (버전 3.9 이상)

@lru_cache는 LRU(least recently used) 알고리즘을 적용하여 일부를 캐싱한다. (버전 3.2 이상)

```
from functools import lru_cache

@lru_cache(None) # 함수에 캐싱 데코레이터를 추가 (크기 무제한)
def trianglepath(y, x):

    ……

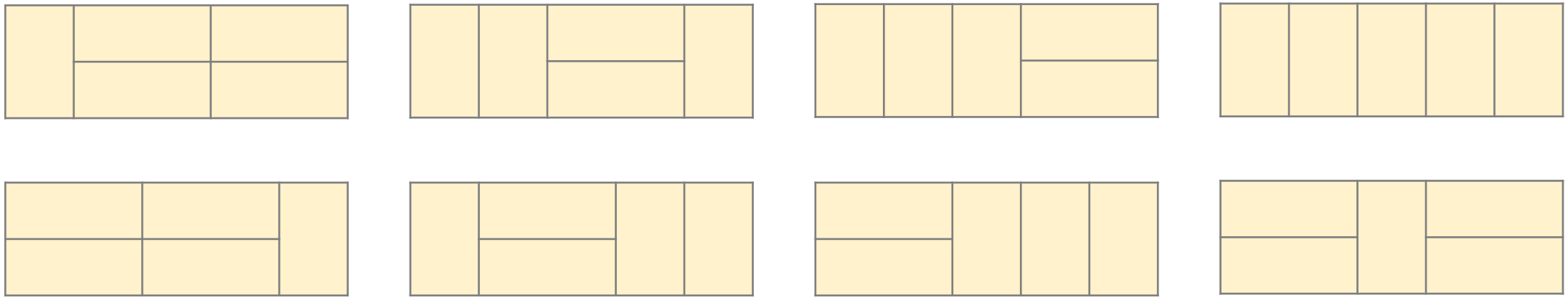
    trianglepath.cache_clear() # 캐시를 비움
```

AL2.4.2: 타일링 방법의 수 세기 (ID: TILING2)

문제 $2 \times n$ 크기의 사각형을 2×1 크기의 사각형으로 빈틈없이 채우는 경우의 수를 구하는 프로그램을 작성하세요. 타일들은 서로 겹쳐서는 안되고, 90도로 회전해서 쓸 수 있습니다.

예를 들어 $n = 5$라고 하면 다음 그림과 같이 여덟 가지의 방법이 있습니다.

경우의 수는 n이 커지면 아주 커질 수 있으므로, 1000000007으로 나눈 값을 대신 출력하세요.



Lec. 04. 동적 계획법 (1)

입력 입력의 첫 줄에는 테스트 케이스의 수(C <= 50)가 주어집니다. 그후 C줄에 각각 1개의 자연수로 n(1 <= n <= 100)이 주어집니다.

출력 각 테스트 케이스마다 한 줄에 경우의 수를 1000000007로 나눈 나머지를 출력합니다.

예제 입력

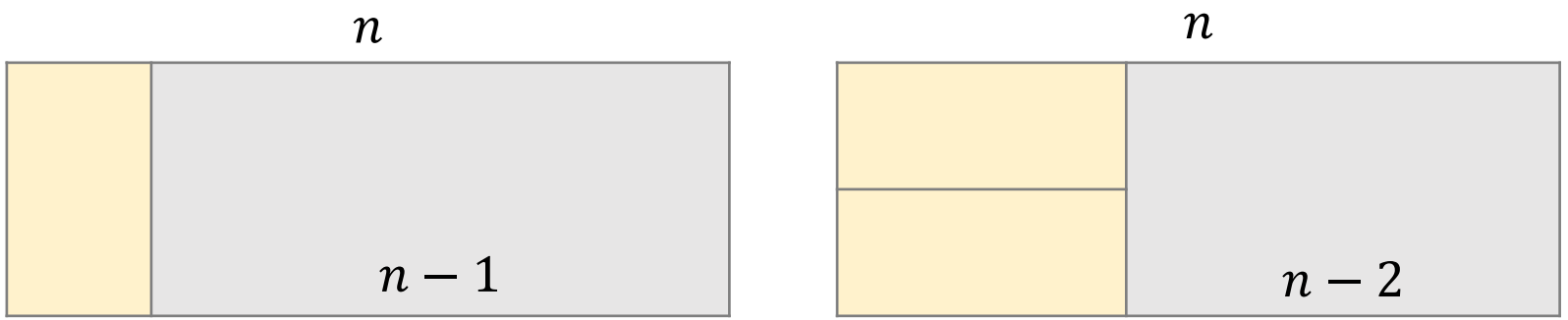
```
3
1
5
100
```

예제 출력

```
1
8
782204094
```

완전 탐색(분할 정복)으로 시작하기

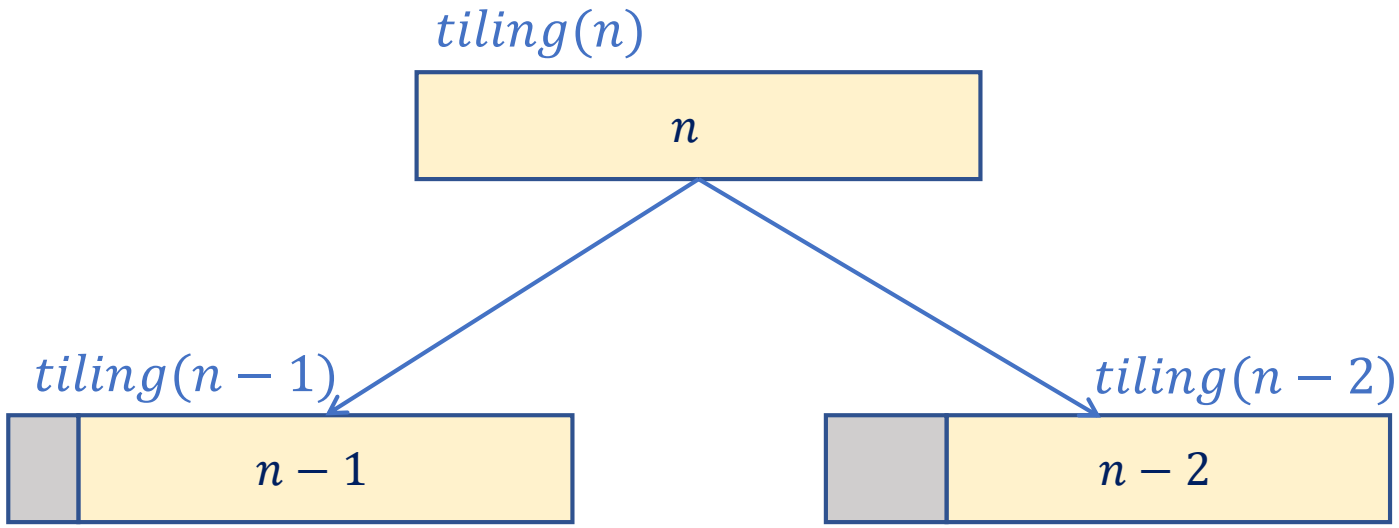
사각형의 맨 왼쪽을 타일로 채우고, 나머지를 재귀 호출하는 전략



경우의 수를 세기 위해 반드시 만족해야 하는 조건

- 위 두 가지 분류는 타일링하는 방법을 모두 포함(누락되는 경우가 없음)
- 위 두 가지 분류에 모두 포함되는 방법은 없음(중복으로 세지 않음)

$tiling(n)$: $2 \times n$ 크기의 사각형을 타일로 덮는 방법의 수



$$tiling(n) = tiling(n-1) + tiling(n-2)$$

$$tiling(1) = 1, tiling(2) = 2$$

초항만 다른 n번째 피보나치 수열

n번째 피보나치 수: 재귀

$$F_0 = 0$$
$$F_1 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

```python
def fibo(n):
    if n <= 1:
        return n
    else:
        return fibo(n - 1) + fibo(n - 2)
```

중복 부분 문제가 발생

n번째 피보나치 수: 메모이제이션

```
# n번째 피보나치 수: 메모이제이션
def fib1(n):
    global cache
    if n <= 1:
        return n
    elif cache[n] != -1:
        return cache[n]
    else:
        cache[n] = fib1(n - 1) + fib1(n - 2)
        return cache[n]
```

```
cache = [-1] * (n + 1)
```

n번째 피보나치 수: 태뷸레이션

```python
# n번째 피보나치 수: 태뷸레이션
def fib2(n):
    dp = [0, 1]
    for _ in range(2, n + 1):
        dp.append(dp[-1] + dp[-2])
    return dp[n]
```

불필요한 테이블을 사용: 공간 복잡도 개선 필요

n번째 피보나치 수: 태블레이션 + 공간 복잡도 개선

```python
# n번째 피보나치 수: 태블레이션 + 공간 복잡도 개선
def fib3(n):
    if n <= 1:
        return n
    else:
        a, b = 0, 1
        for _ in range(2, n + 1):
            a, b = b, a + b
        return b
```

시간 복잡도: $O(n)$, **공간 복잡도:** $O(1)$

이것보다 더 효율적일 수 있을까?

행렬 거듭제곱을 이용한 n번째 피보나치 수

피보나치 수열의 연속된 두 항을 열 벡터로 표현하고, 이를 행렬 연산으로 표현해보자.

$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix}$$

$$F_n = F_{n-1} + F_{n-2}$$
$$F_{n-1} = F_{n-1}$$

$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-2} \\ F_{n-3} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-3} \\ F_{n-4} \end{pmatrix} = \cdots$$

$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

분할정복을 이용한 거듭제곱으로 $O(\lg n)$에 계산 가능

Lec. 04. 동적 계획법 (1)

```
# n번째 피보나치 수: 행렬 거듭제곱을 이용 O(lgN)
def fib4(n):
    if n <= 1:
        return n
    else:
        A = [[1, 1], [1, 0]]
        return mpow(A, n - 1)[0][0]
```

$(0,0)$번째 원소가 F_n임

분할정복을 이용한 행렬 거듭제곱 알고리즘

```
'''
P.4.2. 타일링 방법의 수 세기 (ID: TILING2)
'''
for _ in range(int(input())):
    n = int(input()) # 사각형의 크기(2 * n)
    print(tiling(n))
```

```python
def tiling(n):
    if n <= 2:
        return n
    else:
        a, b = 1, 2
        for _ in range(3, n + 1):
            a, b = b, a + b
        return b % MOD
```

초항이 다름에 유의

큰 수의 덧셈을 여러 번 한 후에

모듈러 연산을 적용 (느림. 이유는?)

모듈러 연산을 적용한 후에 덧셈을 하므로

큰 수의 덧셈이 사라짐 (빠름. 이유는?)

```python
for _ in range(3, n + 1):
        a, b = b, (a + b) % MOD
    return b
```

모듈러 연산의 성질: *modular arithmetic*

모듈러 연산은 아래와 같은 분배 법칙이 성립하므로, 이를 연산 과정에서 잘 활용해야 한다.

$$(A \pm B) \% M = (A \% M \pm B \% M) \% M$$

$$(A \times B) \% M = (A \% M \times B \% M) \% M$$

빠른 행렬 거듭제곱을 이용하는 솔루션에 모듈러 연산을 적용하여 문제를 풀어보시오.

Lec. 04. 동적 계획법 (1)

```python
def mmult(A, B):
    N, K, M = len(A), len(A[0]), len(B[0])
    C = [[0] * M for _ in range(N)]
    for i in range(N):
        for j in range(M):
            for k in range(K):
                C[i][j] = (C[i][j] + (A[i][k] * B[k][j])) % MOD
    return C
```

```
C[i][j] += (A[i][k] * B[k][j])) % MOD
```

오류: 이유는?

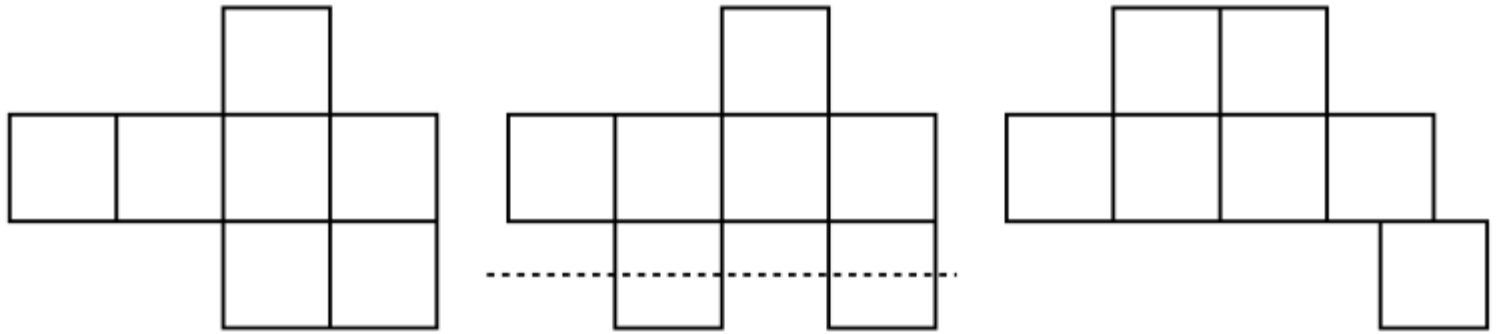
이 부분에 유의할 것

```python
# 행렬 거듭제곱을 이용 O(lgN)
def tiling(n):
    if n <= 2:
        return n
    else:
        A = [[1, 1], [1, 0]]
        return mpow(A, n)[0][0]
```

AL2.4.3: 폴리오미노 (ID: POLY)

문제 정사각형들의 변들을 서로 완전하게 붙여 만든 도형들을 폴리오미노(Polyomino)라고 부릅니다. n개의 정사각형으로 구성된 폴리오미노들을 만들려고 하는데, 이 중 세로로 단조(*monotone*)인 폴리오미노의 수가 몇 개나 되는지 세고 싶습니다. 세로로 단조라는 말은 어떤 가로줄도 폴리오미노를 두 번 이상 교차하지 않는다는 뜻입니다.

예를 들어 그림 (a)는 정상적인 세로 단조 폴리오미노입니다. 그러나 (b)는 점선이 폴리오미노를 두 번 교차하기 때문에 세로 단조 폴리오미노가 아닙니다. (c)는 맨 오른쪽 아래 있는 정사각형이 다른 정사각형과 변을 완전히 맞대고 있지 않기 때문에 폴리오미노가 아닙니다.



Lec. 04. 동적 계획법 (1)

입력 입력의 첫 줄에는 테스트 케이스의 수 C ($1 \le C \le 50$)가 주어집니다. 그 후 각 줄에 폴리오미노를 구성할 정사각형의 수 n ($1 \le n \le 100$)이 주어집니다.

출력 각 테스트 케이스마다, n개의 정사각형으로 구성된 세로 단조 폴리오미노의 수를 출력합니다. 폴리오미노의 수가 10,000,000 이상일 경우 10,000,000으로 나눈 나머지를 출력합니다.

예제 입력	예제 출력
3 2 4 92	2 19 4841817

완전 탐색(분할 정복)에서 시작하자.

어떤 순서대로 폴리오미노를 구성하는 정사각형들을 붙여 나갈까?

중요한 단서: 세로 방향 단조

각 가로줄에 포함된 정사각형들은 항상 일렬로 연속해 있다.

각 가로줄마다 넣을 정사각형의 개수를 결정하고,

적절히 왼쪽-오른쪽으로 밀어주면 모든 폴리오미노를 만들 수 있다.

n개의 정사각형으로 만들 수 있는 폴리오미노의 수:

첫 줄에 1개일 경우의 수 + 첫 줄에 2개일 경우의 수 + ... + 첫 줄에 n개일 경우의 수

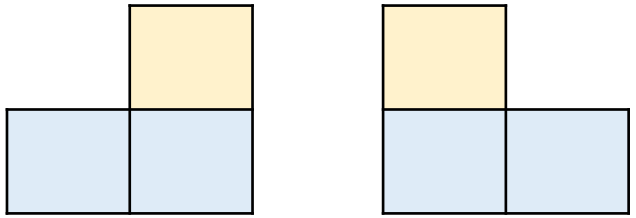
첫 줄에 i개의 정사각형이 속한 폴리노미오의 개수 구하기

나머지 $(n-i)$개로 구성된 폴리오미노를 만들어서(재귀 호출), 둘째 줄에 적절히 붙여준다.

하지만, 둘째 줄에 붙일 수 있는 경우의 수는 둘째 줄에 포함된 정사각형의 수와 상관이 있다.

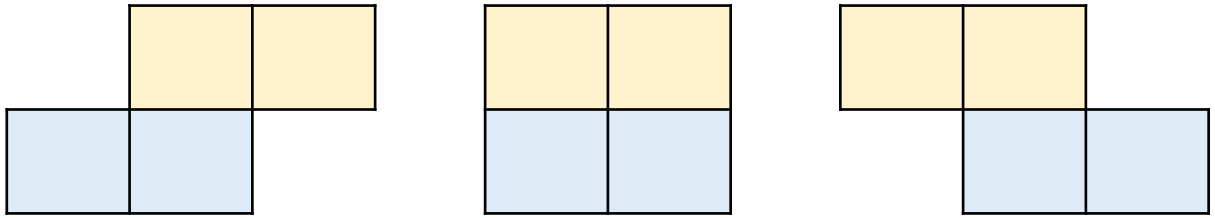
첫 줄에 하나, 둘째 줄에 두 개인 경우:

두 가지 경우의 수가 존재

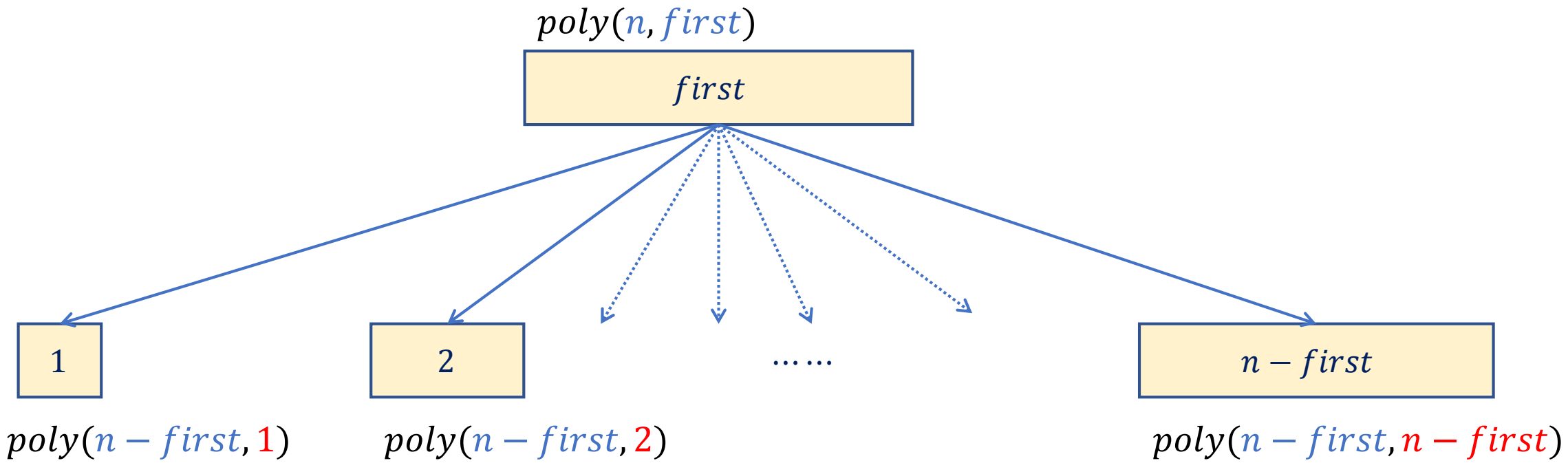


첫 줄에 두 개, 둘째 줄에 두 개인 경우:

세 가지 경우의 수가 존재



$poly(n, first)$: n개의 정사각형을 포함하되, 첫 줄에 $first$개의 정사각형이 들어가는 폴리노미오의 수

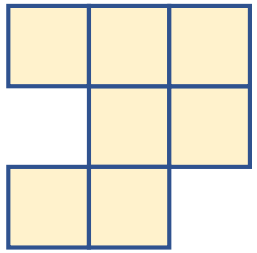


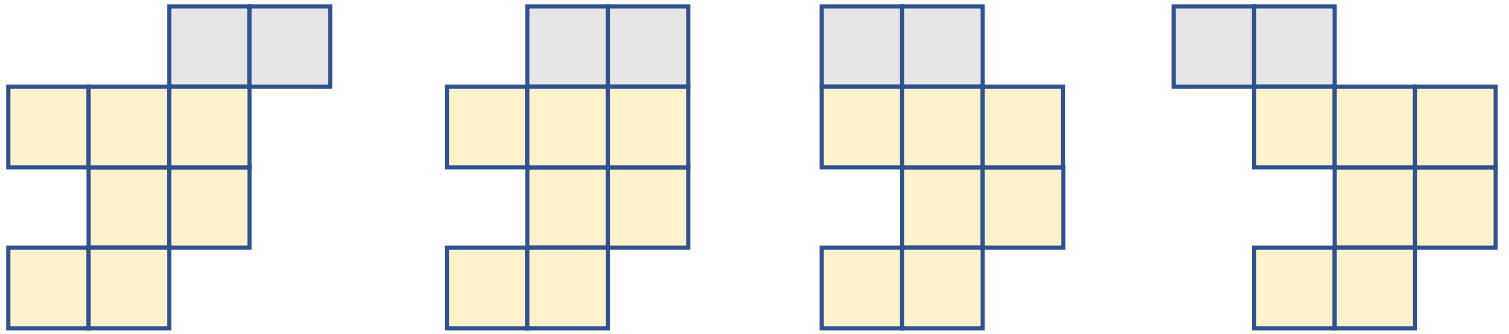
남은 정사각형$(n - first)$으로 만들 수 있는 폴리노미오의 개수:

$$poly(n - first, 1) + poly(n - first, 2) + \cdots + poly(n - first, n - first)$$

남은 정사각형으로 만든 폴리오미노를 첫 줄에 붙이는 경우의 수를 고려:

아홉 개의 정사각형으로 폴리오미노를 만들되, 맨 윗 줄에 두 개를 두고,
나머지 일곱 개의 정사각형으로 만든 폴리오미노가 오른쪽과 같을 때,
아래와 같이 네 가지의 형태로 붙일 수 있다.





$first = 2$

$second = 3$

첫 줄에 $first$개, 나머지 정사각형으로 만든 폴리노미오의 첫 줄에 $second$개가 있을 때,
이들을 붙일 수 있는 방법의 수: $first + second - 1$

Lec. 04. 동적 계획법 (1)

폴리노미오의 개수에 대한 점화식:

$$poly(n, first) = 1, \quad \text{if } n = first$$

종료 조건: 모든 정사각형을 첫 줄에 놓을 때

$$poly(n, first) = \sum_{second=1}^{n-first} (first + second - 1) \times poly(n - first, second)$$

```
MOD = 10000000

'''
P.4.3. 폴리오미노 (ID: POLY): 분할 정복
'''
for _ in range(int(input())):
    n = int(input()) # 폴리오미노를 구성할 정사각형의 수
    ret = 0
    for i in range(1, n + 1): # 첫 줄에 놓을 수 있는 모든 경우의 수를 합한다.
        ret = (ret + poly(n, i)) % MOD
    print(ret)
```

```python
# n개의 정사각형으로 이루어졌고, 맨 위 가로줄에 first개의
# 정사각형을 포함하는 폴리노미오의 수를 반환한다.
def poly(n, first):
    if n == first: # 첫 줄에 모두 다 놓으면
        return 1 # 경우의 수는 하나
    else:
        # 나머지로 만들 수 있는 경우의 수와 합칠 수 있는 경우의 수의 곱을
        # 모두 더한다.
        ret = 0
        for second in range(1, n - first + 1):
            ret = (ret + (second + first - 1) * poly(n - first, second)) % MOD
        return ret
```

폴리오미노: 메모이제이션

```
# 폴리오미노: 메모이제이션
def poly(n, first):
    global cache
    if n == first:
        return 1
    elif cache[n][first] != -1:
        return cache[n][first]
    else:
        ret = 0
        for second in range(1, n - first + 1):
            ret = (ret + (second + first - 1) * poly(n - first, second)) % MOD
        cache[n][first] = ret # 계산한 결과를 캐시에 저장한다.
    return cache[n][first]
```

```
cache = [[-1] * (i + 1) for i in range(n + 1)]
```

캐시 테이블에 저장된 데이터를 통해 태블레이션을 계획해보자.

$first$는 1부터 시작하고,

마지막에 1은 캐시에 저장하지 않았으므로

```
for i in range(n):
    print(*cache[i][1:i], 1)
```

```
1
1 1
3 2 1
10 5 3 1
33 16 7 4 1
107 52 22 9 5 1
344 168 71 28 11 6 1
1103 540 229 90 34 13 7 1
3535 1732 736 290 109 40 15 8 1
```

폴리오미노: 태블레이션

```
def poly(n):
    dp = [[0] * (i + 1) for i in range(n + 1)]
    for i in range(1, n + 1): # bottom-up
        for first in range(1, i + 1):
            if i == first:
                dp[i][first] = 1
            else:
                ret = 0
                for second in range(1, i - first + 1):
                    ret = (ret + (second + first - 1) * dp[i - first][second]) % MOD
                dp[i][first] = ret
    return sum(dp[n]) % MOD # n개로 만들 수 있는 경우의 수를 모두 더한 값을 반환
```

카탈랑 수: *Catalan number*

경우의 수를 구하는 여러 조합 문제에서 자주 등장하는 카탈랑 수를 통해 비슷한 DP를 연습해보자.

카탈랑 수가 등장하는 조합 문제:

- 괄호 문제: 괄호를 올바르게 짝을 맞추는 경우의 수 문제
- BST 문제: 이진 검색 트리의 경우의 수 문제
- 스택 순열 문제: 스택을 통과하여 만들 수 있는 순열의 경우의 수 문제
- 연쇄 행렬 곱셈 문제: 행렬의 곱셈 순서를 만들 수 있는 경우의 수 문제
- 삼각형 분할 문제: $n+2$개의 정점으로 이루어진 다각형을 삼각형으로 분할하는 방법의 수

Lec. 04. 동적 계획법 (1)

$$C_0 = 1$$

$$C_n = \sum_{i=0}^{n-1} C_i \times C_{n-1-i}, n \geq 1$$

카탈랑 수 C_n을 1,000,000,007로 나눈 값을 구하시오.

1. **재귀로 풀기**
2. **메모이제이션 적용하기**
3. **태뷸레이션 적용하기**
4. **일반항 찾아보기**

$C_0 = 1$

$C_1 = C_0 \times C_0 = 1$

$C_2 = C_0 \times C_1 + C_1 \times C_0 = 2$

$C_3 = C_0 \times C_2 + C_1 \times C_1 + C_2 \times C_0 = 5$

$C_4 = C_0 \times C_3 + C_1 \times C_2 + C_2 \times C_1 + C_3 \times C_0 = 14$

$C_5 = 42$

…

$C_{10} = 16{,}796$

…

$C_{20} = 6{,}564{,}120{,}420$

…

$C_{50} = 1{,}978{,}261{,}657{,}756{,}160{,}653{,}623{,}774{,}456$

…

Any Questions?

